

Tiesverse Admin App

One Django admin console that manages three independent public properties — each on its own database and its own Node.js + React stack. This document maps the components, where data lives, and which service every feature talks to.

Admin Django

Public sites Node.js + React

Admin host AWS EC2 · free tier

Web host Hostinger

● Turso

● Cloudflare

● Supabase

Status Draft v0.1

Audience Dev team

Scope Admin console + data flow

Repo single GitHub repo (admin backend)

01 System context

What the admin is, who uses it, and how it relates to the three public sites.

The **Admin App** is a Django application living in a **single GitHub repo**, deployed to an **AWS EC2 free-tier** instance. It is the one back-office where staff manage content for three otherwise-independent public properties:

tiesverse.com	◆ SUPABASE
Main landing page. Events, articles, YouTube, workshops, team, guests & webinar listings.	
career.tiesverse.com	● CLOUDFLARE
Careers page. Open/closed positions, applicant enrollment, offer-letter generation.	
webinar.tiesverse.com	■ TURSO
Webinar property. Managing list & scheduling upcoming events (date / time).	

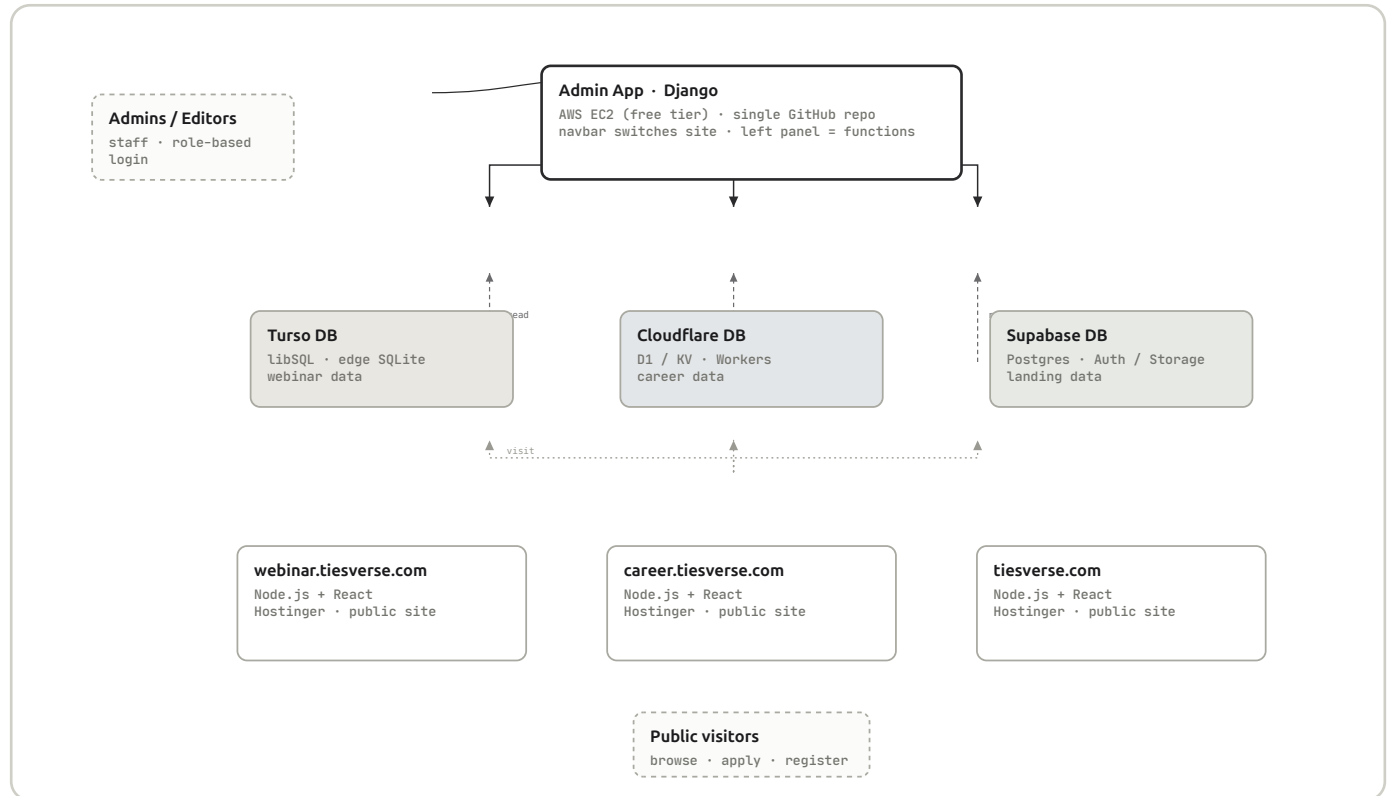
KEY IDEA

Each public site owns **its own database and its own Node.js + React codebase** (hosted on Hostinger). The admin does not replace those sites — it is a **privileged writer** that pushes content into each site's database, while each public site **reads** from its own database to render pages.

Admin → **writes** to Turso, Cloudflare & Supabase · **Public sites** → **read** their own DB · **Visitors** only ever touch the public sites.

02 Architecture & data flow

The single most important diagram: one admin, three databases, three public read-paths.



— Admin writes / manages (solid) - - - Public site reads its DB (dashed) Visitor traffic (dotted)

NOTE

Webinar & career public sites also **write** to their DB for visitor actions (registration forms, job applications). Those write-paths belong to the **public Node apps**, not the admin — shown as the public-site → DB legs. The admin reads that same data back (e.g. the applicant list feeding the enrollment tracker).

03 Deployment topology

Where each piece runs and how the repo is organised.

Admin backend

■ AWS EC2 • FREE

- › ■ **DJANGO** app + admin API, **one GitHub repo**
- › Serves the admin SPA / templated UI
- › Holds write credentials for all three managed DBs
- › Gunicorn + Nginx reverse proxy on the instance
- › Env-scoped secrets: `TURSO_*`, `CF_*`, `SUPABASE_*`

Public web

■ HOSTINGER

- › Three independent **Node.js + React** apps
- › `tiesverse.com` → Supabase
- › `career.tiesverse.com` → Cloudflare
- › `webinar.tiesverse.com` → Turso
- › Each holds read (+ public-write) creds for its own DB only

Managed databases

Turso

■ WEBINAR

libSQL / edge SQLite. Webinar events & schedule.

Cloudflare

● CAREER

D1 / KV via Workers. Positions, applicants, offers.

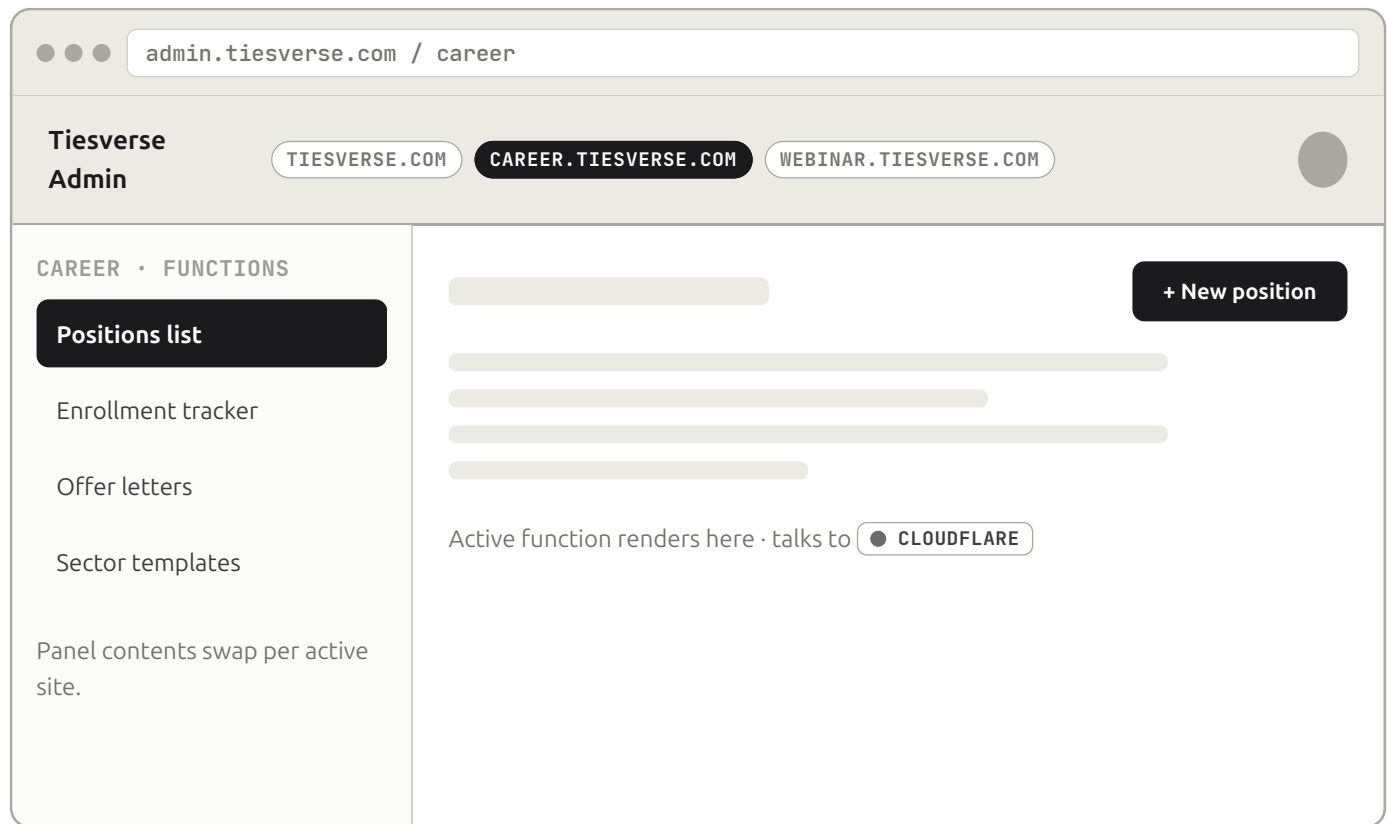
Supabase

◆ LANDING

Postgres + Storage. All landing-page collections.

04 Admin shell

Top navbar switches between the three sites; the left panel shows that site's functions.
The main area renders the active function.



Function panel per site

Landing

◆ SUPABASE

- › Events · Articles · YouTube
- › Workshops · Team
- › Guests · Webinar listings

Career

● CLOUDFLARE

- › Positions list (open/closed)
- › Enrollment tracker
- › Offer letters · Templates

Webinar

■ TURSO

- › Managing list
- › New upcoming event (date / time)

05 Auth & RBAC

Django authentication with role groups. A super admin sees everything; per-site editors are scoped to one property.

Login is handled by **Django auth** (session or JWT). Each user belongs to a **role group**; middleware gates both the navbar sites and the API routes. On login, the user is routed to the first site they can access.

Role	Landing Supabase	Career Cloudflare	Webinar Turso	Generate offers	Users & roles
Super Admin	✓	✓	✓	✓	✓
Landing Editor	✓	—	—	—	—
Career Editor	—	✓	—	✓	—
Webinar Editor	—	—	✓	—	—

Enforced twice: UI hides out-of-scope sites/functions, and every API route re-checks the role server-side. Never trust the client.

06 Webinar module

Manage the webinar list and schedule upcoming events. Admin writes to Turso; the public webinar site reads it.

MODULE 06 Webinar

TURSO • LIBSQL

Managing list

Browse / edit / delete all webinars (past & upcoming). Reorder & mark status.

New upcoming event

Create an event with date & time, speaker and registration link.

DATA webinars table

TURSO

id

title

speaker

date

time + tz

registration_link

cover_url

status upcoming|past

created_by

ROUTES Django admin API → Turso

METHOD	ROUTE	PURPOSE
GET	/api/webinar/events	List (managing list)
POST	/api/webinar/events	Create upcoming event (date/time)
PATCH	/api/webinar/events/:id	Edit details / change status
DEL	/api/webinar/events/:id	Remove webinar

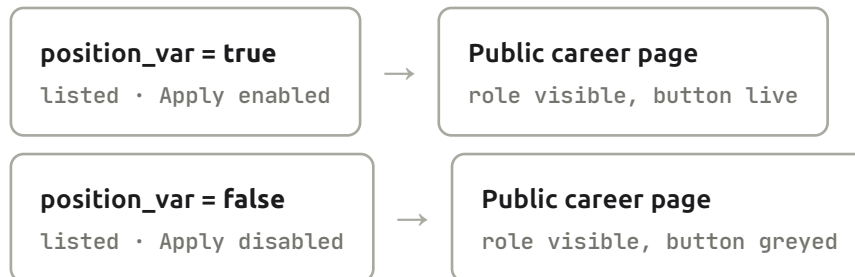
Out of admin scope: the public webinar site (Node) handles visitor register form-fills and "add to calendar", writing directly to Turso. The admin reads those registrations back.

07 Career module

Positions with an open/closed flag, an applicant enrollment tracker, and template-driven offer-letter generation. Admin ↔ Cloudflare.

7.1 Positions list & position_var

`position_var` is a boolean on each position. **true = open** → the role shows on the public career page with a clickable **Apply** button. **false = closed** → the role still appears, but the Apply button is disabled. Editors toggle it from the positions list.



DATA positions

id	title	department	sector tech content design...	location	employment_type
salary_range	description	position_var bool			

7.2 Enrollment tracker

Per position, see everyone who applied and move them through stages. Applicants are created by the **public career site** (apply form → Cloudflare); the admin reads & updates them.

DATA applicants

id	position_id → fk	name	email	phone	resume_url	cover_note
stage applied→hired	applied_at					

7.3 Offer-letter generator

Select an applicant from the tracker → the system picks the template that matches the position's **sector** → generates a filled letter. Templates live in a `/templates` dir as PDFs, converted to HTML by a GitHub repo tool, then merged with the applicant's details.

- 1 Editor opens the **enrollment tracker** for a position and **selects an applicant**.
- 2 Clicks **Generate offer**. Backend reads the applicant + parent position.
- 3 Picks the template by **sector** — tech / content / design ... from `/templates`.

4

PDF template → **HTML** via the conversion tool (GitHub repo).

5

Merge fields: **name, role, salary, start date, location** → preview.

6

Editor reviews & **exports / sends** the final PDF. Record saved to Cloudflare.**DATA offers & templates**

offer.id

applicant_id

template_sector

role_title

salary

start_date

generated_pdf_url

status draft|sent|signed

ROUTES Django admin API → Cloudflare

METHOD	ROUTE	PURPOSE
GET	/api/career/positions	List positions
POST	/api/career/positions	Create position
PATCH	/api/career/positions/:id	Edit / toggle position_var
GET	/api/career/positions/:id/applicants	Enrollment tracker
PATCH	/api/career/applicants/:id	Move stage
GET	/api/career/templates	List sector templates
POST	/api/career/offers/generate	Build offer from template + applicant

08 Landing module

Seven content collections behind tiesverse.com. Each is plain CRUD against Supabase.

MODULE 08 **Tiesverse landing**

◆ SUPABASE • POSTGRES

EventsEvent management · `events`**Articles**Article management · `articles`**YouTube videos**Embedded video list · `youtube_videos`**Workshops**Workshop list · `workshops`**Team**Team members · `team_members`**Guests**Past & upcoming · `guests`**Webinar listings**Past / upcoming · `webinars` (see note)ROUTES **Uniform CRUD per collection → Supabase**

METHOD	ROUTE	PURPOSE
<code>GET</code>	<code>/api/landing/:collection</code>	List items
<code>POST</code>	<code>/api/landing/:collection</code>	Create item
<code>PATCH</code>	<code>/api/landing/:collection/:id</code>	Edit item
<code>DEL</code>	<code>/api/landing/:collection/:id</code>	Delete item

`:collection` ∈ `events` · `articles` · `youtube_videos` · `workshops` · `team_members` · `guests` · `webinars`. Media uploads use Supabase Storage.

DECIDE

The landing page lists **webinars (past/upcoming)**, but the canonical webinar data lives in **Turso**. Choose one: **(a)** the landing site fetches a read-only feed from the webinar property, or **(b)** the admin mirrors a lightweight copy into Supabase on publish. (a) avoids duplication; (b) keeps tiesverse.com self-contained. See §11.

09 Screen wireframes

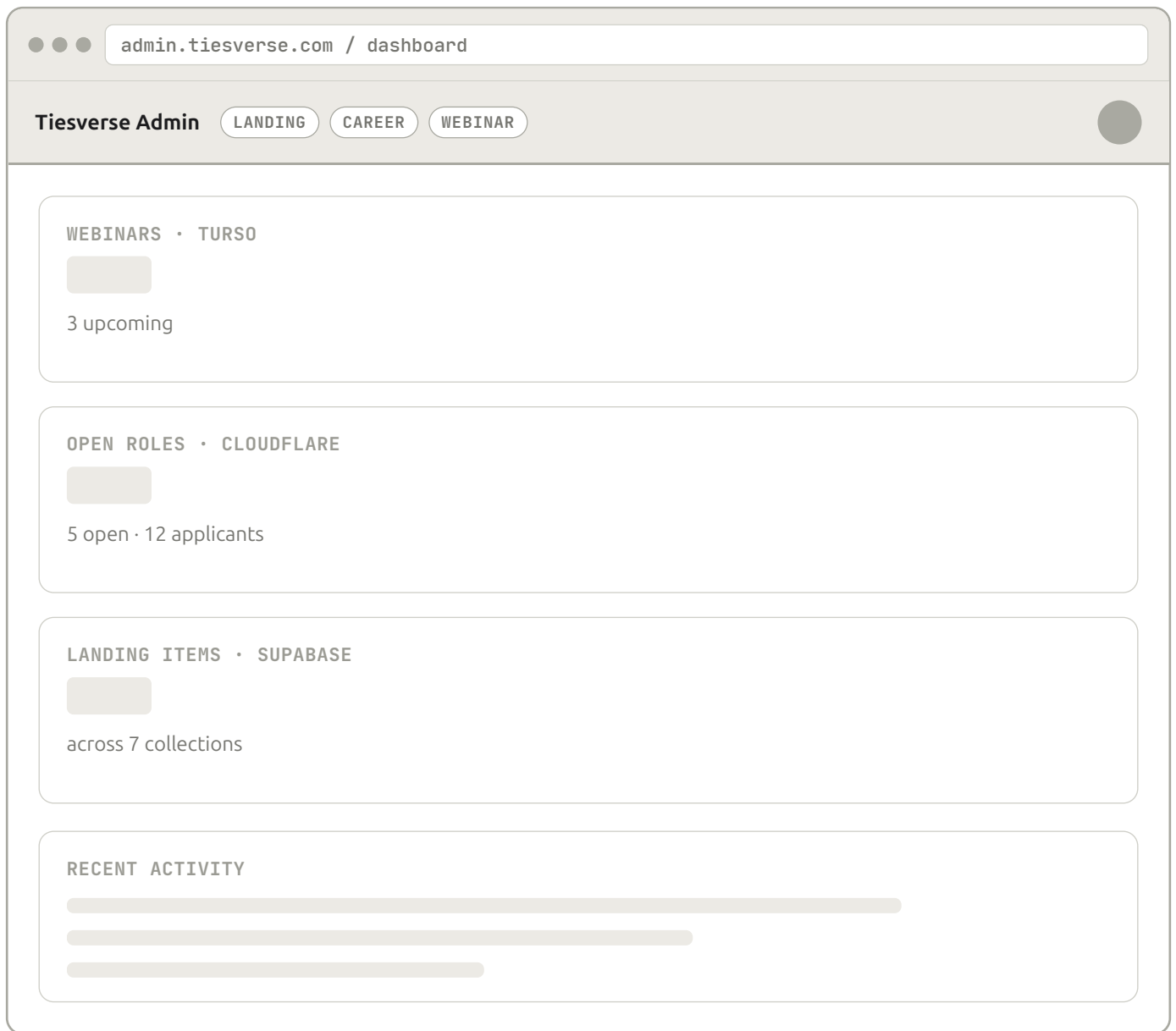
Low-fi layout intent for the priority screens. Greyscale, no branding.

9.1 Login (role-based)

The wireframe shows a login page for 'Tiesverse Admin'. The browser's address bar displays 'admin.tiesverse.com / login'. The login form is centered and contains the following elements:

- A header bar with the text 'Tiesverse Admin'.
- An 'EMAIL' label above a text input field containing 'you@tiesverse.com'.
- A 'PASSWORD' label above a password input field with seven dots.
- A dark 'Sign in' button.
- A message below the button: 'Routed to first permitted site by role'.

9.2 Dashboard / overview



9.3 Career — positions list & open/closed toggle

admin.tiesverse.com / career / positions

+ New position

POSITION	SECTOR	APPLICANTS	POSITION_VAR
Frontend Engineer Engineering · Remote	tech	7	☒ OPEN
Content Writer Content · Hybrid	content	4	☒ OPEN
Product Designer Design · Remote	design	1	☐ CLOSED

Toggle flips position_var on Cloudflare → controls the Apply button on the public career page. Closed roles stay listed.

9.4 Career — enrollment tracker

admin.tiesverse.com / career / positions / frontend-engineer

▼ Frontend Engineer

OPEN

7 applicants

APPLICANT	APPLIED	STAGE	ACTION
A. Rahman a.rahman@mail.com	May 21	INTERVIEW	Generate offer
S. Mehta s.mehta@mail.com	May 19	REVIEW	View
J. Okoro j.okoro@mail.com	May 18	APPLIED	View

Reads /api/career/positions/:id/applicants from Cloudflare. **Generate offer** opens 9.5.

9.5 Career — offer-letter generator

admin.tiesverse.com / career / offers / new?applicant=a-rahman

APPLICANT

A. Rahman

SECTOR TEMPLATE

tech ✓ (auto from position)

content

design

MERGE FIELDS

Role · Frontend Engineer

Salary · ____

Start date · ____

Location · Remote

LETTER PREVIEW · PDF→HTML
template

Save draft

Export PDF

Template chosen by sector → converted PDF→HTML → merge fields filled → export. Saved to offers on Cloudflare.

17 of 19

31/05/26, 3:13 am

10 API surface

Every admin route, the service it reaches, and the role required. All routes are served by the Django backend on EC2.

METHOD	ROUTE	SERVICE	ROLE
POST	/api/auth/login	 DJANGO	Any
GET	/api/dashboard/summary	All three	Any (scoped)
GET	/api/webinar/events	 TURSO	Webinar / Super
POST	/api/webinar/events	 TURSO	Webinar / Super
PATCH	/api/webinar/events/:id	 TURSO	Webinar / Super
GET	/api/career/positions	 CLOUDFLARE	Career / Super
PATCH	/api/career/positions/:id	 CLOUDFLARE	Career / Super
GET	/api/career/positions/:id/applicants	 CLOUDFLARE	Career / Super
POST	/api/career/offers/generate	 CLOUDFLARE	Career / Super
GET	/api/landing/:collection	 SUPABASE	Landing / Super
POST	/api/landing/:collection	 SUPABASE	Landing / Super
PATCH	/api/landing/:collection/:id	 SUPABASE	Landing / Super
POST	/api/users	 DJANGO	Super only

11 Open decisions

Items to confirm before/while building. Each affects the data flow above.

D1

Admin → DB access pattern. Does Django write **direct to each DB** (Turso/Cloudflare/Supabase SDKs), or call each public site's Node API? This doc assumes direct writes (simplest, fewest hops). Through-API gives each site sole ownership of its schema. **Recommend: direct, with shared schema docs.**

D2

Landing webinar listings. Canonical webinar data is in Turso but tiesverse.com lists webinars. Read-only cross-fetch vs. mirror-into-Supabase-on-publish. (See §8.)

D3

Offer PDF→HTML tool. Which GitHub tool converts the sector PDF templates to HTML, and does it run on EC2 at request time or as a one-off build step that stores HTML templates? Affects latency & the / templates layout.

D4

EC2 free-tier limits. Single t2/t3.micro hosts Django + Gunicorn + Nginx. Confirm headroom for PDF generation; consider offloading PDF render to a queue if it spikes memory.

D5

Auth transport. Session cookies (simplest with Django) vs. JWT (needed if the admin SPA and API are on different origins). Pick based on whether admin UI is server-rendered or a separate React app.